

What our returns do, and do not, prove

Weekly research synthesis | 28 September - 4 October 2026 | Q014

Sidiq review status: Not yet reviewed. Repository completion does not establish personal mastery.

The week's conclusion

We can now check a bounded decision calendar, value synthetic splits and cash entitlements, retain historical members and unresolved holdings, and link returns using only available revisions. These are useful accounting and information controls. They are not an investable strategy, authenticated historical dataset or empirical alpha result. The most important unfinished work is at the interfaces: who supplied each input, when its underlying facts were known, and whether entitlement became cash.

This report synthesises the seven daily lessons and today's authorised curriculum revision. It does not repeat the morning's Q010 implementation. The prior weekly boundary is 8228415; eight subsequent commits through 5d33319 were present before this synthesis. All dates below are actual commit dates.

Completed evidence and commit trail

Date / commit	Bounded completion	Main evidence
Sep 28 / cb610d6	Q006a calendar contract	33 dates; 22 sessions; six corruptions rejected
Sep 29 / 3e81408	Q006b calendar API	22 pre-open decisions; 132 core boundaries; nine invalid decisions
Sep 30 / b6da07a	C002 compound savings	66 C++ checks; simple-interest substitution fails
Oct 1 / beb35a4	Q008 action returns	Eight hand examples; 162 exact-ledger comparisons
Oct 2 / a5f9f86	Q009a membership	Eight date sets; 192 event-ledger comparisons
Oct 3 / d3385de	Q009b terminal evidence	Six evidence states; 108 exact-ledger comparisons
Oct 4 / 758d9de	Q010 lagged features	125 exact wealth paths; 420 timing/window cases
Oct 4 / 5d33319	New curriculum and cadence	C++ default; six commit-budget persistence tests

Daily Python test totals were 292, 363, 363, 415, 470, 540 and 602. Today's curriculum commit adds six tests, making the weekly validation total 608. Counts describe executed cases, not coverage or independent economic observations. All prices, balances, identities and returns in the lessons are synthetic. The October-November calendar is a prospective schedule, not realised trading evidence.

Concept 1: separate the clocks

A session is a scheduled exchange day. A decision is when a policy fixes its information set. A fill is an executed transaction with a price and quantity. Checking one does not prove the others. The policy permits a 09:00 New York decision before the 09:30 core open. A rule requiring decisions to occur during core hours would wrongly reject all 22 valid fixture decisions.

For scheduled open O and close C , the project uses $O \leq t < C$. The exact close is outside this predicate; a completed daily bar can still be labelled at C . This convention does not model auctions. A known holiday returns no session. Unknown dates outside coverage raise rather than masquerade as holidays. A valid decision may still lack enough prior sessions or received observations.

The bounded fixture covers October 29-November 30. November 26 is closed; November 27 closes at 13:00 Eastern. Its prior session is November 25. Its core duration is 210 minutes versus 390 on a regular day. A weekdays-only November count incorrectly gives 21 sessions instead of 20. The official NYSE schedule was rechecked on October 4 and agrees with these scheduled facts:
<https://www.nyse.com/trade/hours-calendars>

09:00 New York is 13:00 UTC on October 30 and 14:00 UTC on November 2. Keep the policy in its named zone; do not freeze the UTC hour. Calendar validity still says nothing about halts, emergency closures, actual liquidity or input availability. Publication, receipt and processing delay must all be respected.

Concept 2: value the same holding at both endpoints

For one opening share, opening raw price P_0 , closing raw price P_1 , ending shares per opening share s , and cash entitlement per opening share C , total return is $(sP_1+C)/P_0-1$. Consistent units do the work.

Synthetic case	Correct accounting	Wrong conclusion exposed
Split: 100 to 50; $s=2$	$2*50=100$; return 0%	Raw-price change says -50%
Split plus dividend: $P_1=49$, $C=2$	$2*49+2=100$; return 0%	Multiplying C by s again says +2%
Savings: 1000 at 5% for two years	$1000*1.05*1.05=1102.50$	Simple interest says 1100

Q008 values an entitlement at par, without reinvestment, settlement, fees, tax or FX. Required inputs prevent omission; they cannot detect already adjusted prices or falsely declared units. C002 assumes constant nonnegative annual rates and whole years. Its float result is neither an exact money ledger nor a bank-product quote. Both lessons require the financial convention before the calculation.

Concept 3: eligibility, ownership and evidence differ

Membership uses entry $\leq$ date $<$ exit. On synthetic January 2, OLD, RETURN and STAY belong. Filtering that set by the January 8 roster removes OLD incorrectly. We established a selection error; with no returns attached to this fixture, neither the sign nor size of performance bias was measured.

Effective-date history does not establish knowledge at a decision. A later correction to an earlier membership interval may change the past. Stable identities, complete history and publication vintages still require upstream evidence. The current API cannot authenticate them.

A holding also survives an eligibility exit. Ten shares priced at 20 have reference value 200. With complete terminal cash of 5 per reference share, entitlement is 50 and return is $50/200 - 1 = -75\%$. Missing complete proceeds yield unknown value and return. Supported complete zero yields 0 and -100% . A known instalment of 5 is insufficient if further payments remain uncertain. The fixture's partial recovery means a COMPLETE payment below the reference value, not incomplete payment evidence.

Wrongly dropping the holding after membership exit removes the accounting problem from sight. Replacing unknown proceeds with zero fabricates a -100% result. Neither is justified. Q009b retains the position and evidence but supplies no settlement, portfolio aggregation or as-known-at event model.

Concept 4: compound only information that was available

Geometric linking multiplies gross factors: $G = \text{product}(1 + r_i)$, $R = G - 1$. Starting at 100, $+10\%$ gives 110; then -10% loses 11, leaving 99. The linked return is -1% , while addition incorrectly gives zero. Each input must describe a complete, consistently defined preceding-session return.

At the synthetic November 4 decision, Q010 selects $+10\%$ and -10% . Adding later returns and a $+50\%$ revision published November 5 leaves that full earlier result unchanged across 24 input orderings. On November 5, the same historical window can legitimately use the revision and become $+65\%$. One microsecond of processing delay excludes the revision arriving exactly at that later decision, restoring -1% . A missing required interval stays unknown; an older row cannot fill its place.

These controls refute two shortcuts: summing returns and using the latest revision without its publication time. They do not authenticate publication times. Valid-future-extension invariance assumes unchanged original records and valid new rows; malformed future rows can still raise errors. Linking Q008's entitlement returns also does not prove reinvestment was feasible. That needs an explicit cash/receivable and execution model. No rank, target, benchmark or strategy was added today.

What code-quality evidence actually exists

Weekly checks ran on the existing local main checkout at 5d33319. make check passed: 608 Python tests; Ruff lint and formatting for 49 files; strict mypy for 21 package source files; 66 C++ checks and exact demonstration output. make -B cpp-check then rebuilt both executables with C++17 and -Wall -Wextra -Wpedantic -Werror. Dependency consistency also passed. No new dependencies were added.

Eleven JSON artifacts were each replayed twice into ignored scratch output and matched the archived bytes exactly: calendar audit/decisions, actions, membership, terminal, lagged returns, manifests, purging, data access, timezone folds and ingestion. Two freshly built C++ demo runs matched expected.txt. Q003's replay verified its four original hashes; a hash identifies bytes, not truth or a licence.

Independent references improve confidence: exact Fraction accounting differs from the float code; membership's entry/exit ledger differs from interval containment; Q010's oracle uses explicit indices and availability filtering instead of production history/snapshot helpers. Their bounded input sets are not exhaustive. No line/branch coverage, statistical power, latency benchmark or general mutation score was measured. Strict mypy covers src, not every script or test. Passing local checks is not CI.

The authenticated issue read returned zero open issues. Baseline CI for exact SHA 5d33319 completed successfully (run 37204224669). The new synthesis commit's remote/CI outcome must be checked after publication and recorded in the briefing/memory; it is not predicted by this pre-publication report.

Failures and negative results worth retaining

Calendar lessons caught a generator-formatting issue and overlong diagnostic strings. The terminal lesson caught another overlong line. Q010 first expected 20 tiny positive factors to underflow, but 2^{-1060} remains representable. The test expectation was corrected; this was not a production fix. Its fractional return can round to -1 while gross remains positive. Do not infer worthlessness from a rounded percentage. Q010 also required explicit zip strictness and a loop-variable typing repair.

Daily PDF reviews caught spill pages on September 28/29 and October 1/4. Shortening or moving text and repeating extraction plus every-page inspection resolved them. Text equality alone missed layout problems. No failed check was suppressed. This week's report receives its own text and visual review.

Restricted fetch initially failed DNS; the approved retry succeeded. This weekly run's Python GitHub read failed certificate verification; system curl succeeded with TLS verification retained. Employer page refreshes for AQR and Jane Street were inaccessible, so their September 21 evidence stays dated. Daily CRSP methodology links returned 404 on October 1; an October 4 glossary entry had no useful text. Neither failed retrieval supplied evidence. There is still no empirical performance result to report.

Employer-skill connections, with limits

The September 21 role study sampled 49 postings at eight employers; it mixed senior and junior roles. It is a dated, purposive sample, not a current vacancy count or a measured hiring probability. `career_research/role_skill_analysis.md` and `role_skill_matrix.csv` preserve sources and methodology.

AQR's recorded data-scientist requirements connect data validation, tests, Git and CI/CD to this week's boundary checks and replay evidence. Jane Street's recorded research-engineering requirements connect reproducible code and infrastructure to immutable results and retained selected observations. The current pages could not be refreshed on October 4; no continuing vacancy is asserted.

AQR source: https://careers.aqr.com/jobs?gh_jid=8088184 Jane Street source: <https://www.janestreet.com/join-jane-street/apply/7437779002>

The recorded Citi model-validation role connects independent benchmarks and numerical accuracy to the exact ledgers, compounding counterexample and documented float limitations. The recorded IMC C++ role motivates clear interfaces, compilation and invalid-input handling. These are our skill mappings, not employer endorsements; a toy calculation does not substitute for production or commercial tenure.

Citi source:

<https://jobs.citi.com/job/new-york/model-validation-2nd-lod-sr-lead-analyst/287/100430304704> IMC source: <https://job-boards.eu.greenhouse.io/imc/jobs/4945420101>

Communication practice is concrete: explain why a supported zero differs from no evidence, derive the loss on a 100-to-110-to-99 path, then name assumptions before discussing tests. Personal review remains Not yet reviewed; neither a passing build nor an automated note proves independent explanation.

Evidence-based planning changes

`QUANT_CURRICULUM.md` and `CURRICULUM_BACKLOG.md` govern October 5-December 31. C++ becomes the default for new educational projects; the tested Python modules remain reference and verification material. The old October 5 Q011 ranking date and Wednesday-only rule no longer dispatch tasks.

C002 exposed a precise prerequisite: an int parameter cannot detect a fractional value converted before the function call. Split C003 into C003a, strict whole-year text parsing, and C003b, the later principal/rate parser and full CLI. Next week develops only C003a slowly. C003 remains incomplete until both children pass. Vectors, price returns and sample statistics remain C004/C005, introduced later after parsing and function prerequisites; do not stack them into this week's sessions.

Split deferred Q005b into a source/units/knowledge contract (Q005b1) and later two-batch adapter replay (Q005b2). The contract must distinguish effective dates from publication, complete proceeds from instalments, receivables from settled cash, and linkable interval basis. Integration still depends on Q013 and audited access before empirical use. This is dependency planning, not a second daily workload.

Next week: one bounded concept per day

Date	C003a focus	Acceptance or stopping point
Oct 5	Text versus whole-year values	Hand-classify 2, 2.5, -1, blank and 2x; explain conversion loss
Oct 6	Define a years-only grammar	Nonnegative decimal digits, full consumption, range; choose whitespace policy
Oct 7	Implement one parser function	Explain branches line by line; valid/invalid examples; retain C002
Oct 8	Boundary tests and Python oracle	Zero, maximum int, overflow, trailing text; exact accepted values
Oct 9	Deliberate fractional-input failure	Show why truncating 2.5 to 2 changes meaning; repair unclear reasoning
Oct 10	Reproduce and teach back	Same inputs/outputs; exercises and separate answers; no new complexity
Oct 11	Consolidate and weekly synthesis	Review parser evidence; decide whether C003b is ready

A session remains 45-90 minutes. Repeat prerequisites when needed. The October 12 ledger slice is conditional on sufficient understanding of functions and branches; narrow or defer it if those are not ready. Mean/variance and full CLI work are explicit carryover, not silently marked complete. The broad strategy atlas remains a programme direction, not a promise of production depth by New Year.

Publication cadence and questions for Sidiq

From October 5, after taking the shared run lock, run `scripts/daily_commit_budget.py` once per London date and reuse its persisted uniform 0-3 draw. Count actual programme commits already made that day, including manual and weekly publication. The script saves a draw; it does not count commits or enforce the cap. Check Git history and both automation memories before committing. A draw is a ceiling, not permission to split a lesson or manufacture work. Today's draw is not applicable; the rule starts tomorrow.

On zero or exhausted-budget days, save notes, verified PDFs and an integration manifest under `ignored work/pending/YYYY-MM-DD/` without tracked edits. Later integration retains the true work date, revalidates, and uses the actual publication date. Already validated unpushed commits may be retried separately. The weekly synthesis uses the same allowance as that morning. No additional lesson fills a spare slot.

The schedule remains Sundays 16:00 Europe/London, including the October clock change. New programme work stops after December 31 and requests the next scope. No schedule mutation was needed this run.

Input needed before empirical research: existing licensed entitlement or a preferred source/budget, followed by a lawful sample audit. Do not send credentials. The scope question is whether any narrower historical-vintage claim is acceptable if the source cannot support the full claim; no narrowing has been approved. The separate finance project's identity also remains unresolved and is not substituted. No answer is needed to continue the bounded synthetic C++ lessons. If the pace is too fast, identify which prerequisite to repeat; personal review is separate from standing publication authorisation.

Revision questions

1. Can a 09:00 decision be valid while a 09:00 fill is unproved? What else is required?
2. A split changes one share at 100 into two at 49, with 2 cash per opening share. What is the return, and why might that cash still be unavailable for reinvestment?
3. OLD exits the eligible universe. Does that create a sale? What changes if terminal cash is unknown versus explicitly supported complete zero?
4. Why can the same old return window give -1% at one decision and +65% later without look-ahead?
5. Why can a C++ int parameter accept a converted fractional expression, and where must validation occur?

Suggested answers

1. Yes. The policy validates a session and decision clock. A fill additionally needs an execution model, price, quantity, timing, liquidity and cost evidence. Required observations must also be received and processed by the decision; scheduled prior sessions alone cannot supply them.
2. Closing wealth is $2 \times 49 + 2 = 100$, so return is zero. The 2 is entitlement valued at par, not necessarily settled cash. Reinvestment needs its own timing/financing convention and execution evidence.
3. No. Retain the holding until supported transactions or event accounting resolve it. Unknown complete cash gives unknown value/return. Supported complete zero gives zero value and -100%. Do not omit the unresolved holding from an aggregate or treat a partial instalment as complete evidence.
4. The later decision can use a newly available revision. At the earlier decision +10% then -10% produces -1%; after the second return is revised to +50%, $1.10 \times 1.50 - 1 = +65\%$. Filtering by publication and processing time preserves the earlier answer. A newer vintage does not retroactively become known.
5. Conversion can occur before the function body runs, losing the fraction. Validate the original text's grammar, full consumption and range before converting. This is next week's narrow C003a task, not functionality already present in the savings demo.

Reproduction and evidence index

Daily explanations: `research_log/2026-09-28.md` through `research_log/2026-10-04.md`. Daily observed failures and checks: `reports/milestone/2026-09-28-validation.md` through `reports/milestone/2026-10-04-validation.md`. Synthetic inputs, archived outputs and source details: `experiments/calendar`, `corporate_actions`, `membership`, `terminal` and `lagged_returns`. C++ source and hand-authored output: `cpp/projects/savings_growth`. Weekly check details: `2026-10-04-validation.md`.

Reproduce current checks with `make check`, `make -B cpp-check` and `.venv/bin/python -m pip --no-cache-dir check`. Each replayed script accepts `--output PATH`; use ignored scratch output to compare with the archive. Render this note using `scripts/render_note.py`, then verify normalized source text, render every page with Poppler and visually inspect the results.

This synthesis adds explanation and evidence-based planning only. It does not modify the completed experiment results, expand calendar coverage, acquire market data or certify an empirical strategy.